



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Agents for Trading (C177)

Course Code: C177

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	11 August 2026	Initial aligned non-WSQ release with two topics, four connected labs, and a paper-only trading workflow.	Course Trainer

TABLE OF CONTENTS

Introduction.....	5
Course Learning Outcomes.....	5
Before You Start — Preparation.....	5
Topic 01 — From Trading Idea to Quality Market Data.....	6
Chatbot, Workflow, Agent, or Autopilot?.....	7
Seven Building Blocks of a Bounded Trading Agent.....	7
The Research Agent Loop.....	8
A Trading Idea Is Not Yet a Hypothesis.....	8
The HYPER Hypothesis Contract.....	9
Research Evidence: Stronger and Weaker Uses.....	9
An Agent Output Needs a Contract.....	10
What a Daily OHLCV Bar Represents.....	10
Feed Coverage Changes What You Observe.....	11
Time, Session, and Signal Availability.....	11
Corporate Actions and Adjusted Prices.....	12
The Market-Data Contract.....	12
Eight Data-Quality Gates.....	13
Worked Example: Freeze the SPY Research Pack.....	13
Lab 1 — Research an Idea and Freeze a Testable Hypothesis.....	14
Lab 2 — Fetch Market Data and Prove It Is Fit for Purpose.....	17
Recap — From Trading Idea to Quality Market Data.....	20
Topic 02 — The Six-Step Trading Workflow and Staying in Control.....	20
The Six-Step Controlled Trading Workflow.....	21
Rule Specification: Remove Every Hidden Choice.....	21
Look-Ahead Bias: The One-Bar Test.....	22
Backtest Engine: Inputs to Evidence.....	22
Development Window and Out-of-Sample Window.....	23
Read Metrics as a System, Not a Score.....	23
Frictionless Results and More Realistic Results.....	24
Overfitting Red Flags.....	24
The Backtest Audit Trail.....	25
Model Work and Deterministic Work.....	25
Position Size Comes from Risk Budget and Stop Distance.....	25
Worked Position-Size Example.....	26
The Pre-Trade Paper Ticket.....	26
Paper Trading Is Useful and Incomplete.....	27
Human Gate and Safe Submission.....	27

Verify-Then-Trust Checkpoints.....	28
Failure, Pause, Reconcile, and Roll Back.....	28
Lab 3 — Backtest the Rules and Audit the Evidence.....	29
Lab 4 — Size Risk, Approve, and Verify a Paper Trade.....	32
Recap — The Six-Step Trading Workflow and Staying in Control.....	35
Wrap-Up - Your Controlled Trading Research Pack.....	36
Next Steps.....	36
Glossary.....	37

Introduction

AI Agents for Trading is a one-day, beginner-level course about disciplined trading research, not automated profit. You will build a bounded research agent that can organise a hypothesis, call approved Python tools, interpret structured observations, and prepare a paper-order exercise. Calculations and controls remain deterministic, consequential steps require human approval, and the supplied execution path cannot send a live order.

All four labs use one SPY daily-bar moving-average scenario and one C177-trading-agent-pack. Each lab begins from a verified checkpoint and produces versioned JSON, CSV, Markdown, image, or receipt evidence for the next lab. The strategy is deliberately simple so the course can focus on research quality, data lineage, causal testing, risk limits, paper execution, and the verify-then-trust habit.

Course Learning Outcomes

- LO1: Explain how a bounded AI trading agent combines a model, instructions, tools, state, evidence, guardrails, and human control without becoming an autonomous money manager.
- LO2: Research a trading idea with an AI agent and convert it into a structured, falsifiable hypothesis with explicit rules, assumptions, evidence, and rejection criteria.
- LO3: Connect the agent workflow to an authorised market-data source and validate coverage, timestamps, corporate-action adjustments, completeness, consistency, and freshness before use.
- LO4: Define deterministic trading rules, run an honest backtest, compare a benchmark, and audit data leakage, friction assumptions, overfitting, and out-of-sample behaviour.
- LO5: Calculate a bounded position size from account risk and stop distance, then prepare an auditable paper-order ticket with pre-trade limits and human approval evidence.
- LO6: Place and verify a paper-only trade through a controlled agent workflow while applying verify-then-trust checkpoints, logs, stop conditions, and rollback procedures.

Before You Start — Preparation

What you need

- A Windows or Mac laptop with Python 3.11 or newer, Visual Studio Code or Google Colab, and a modern browser.
- The C177 repository and permission to create a local C177-trading-agent-pack folder.
- An OpenAI API key stored only in a local environment variable for the agent labs, or a trainer-provided approved equivalent.

- An Alpaca paper-only account with paper API keys. Never use live-trading credentials in this course.
- Internet access for official documentation and authorised API calls; a supplied synthetic checkpoint is available only as a rejoin fallback.

Verify your setup

Create a virtual environment, install the pinned lab dependencies, copy the example environment file, then run the preflight check. Do not paste keys into chat, code, screenshots, notebooks, or git.

```
python -m venv .venv

# Windows: .\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate

python -m pip install -r labs/resources/requirements.txt

Copy-Item labs/resources/.env.example .env # Windows
# cp labs/resources/.env.example .env      # macOS/Linux

python labs/resources/preflight.py
```

Conventions used in every lab

- Commands are run from the repository root unless a step says otherwise.
- Placeholders such as <OPENAI_API_KEY> and <ALPACA_PAPER_KEY> are replaced only in the local .env file, which git ignores.
- The default OpenAI model for this August 2026 build is gpt-5.6-sol; trainers should recheck the official model guide before delivery.
- ALPACA_PAPER_ONLY must remain true. The supplied client constructs TradingClient(..., paper=True) and contains no live-mode flag.
- All prices, strategy outputs, and paper fills are educational observations rather than recommendations or promises.
- If a critical check reports HOLD, stop and follow the troubleshooting or rejoin path before continuing.

Topic 01 — From Trading Idea to Quality Market Data

AI trading agents | research evidence | structured hypotheses | market-data connections
| data-quality gates

Key concepts

- Bounded agency: The model may research and recommend, but deterministic tools calculate, controls constrain, and a person owns consequential decisions.

- Falsifiable hypothesis: A useful idea states the asset, data, signal, entry, exit, horizon, benchmark, costs, risks, and rejection rule before testing.
- Evidence chain: Every material claim links to a source, timestamp, tool result, assumption, or named owner instead of model confidence.
- Market-data contract: The workflow records provider, feed, symbol, timezone, session, adjustment, interval, coverage, and intended use.
- Quality gates: Schema, chronology, uniqueness, completeness, OHLC consistency, volume, freshness, and corporate actions are checked before analysis.
- Human control: The agent stops when evidence is missing, conflicts remain, data quality is weak, or an action exceeds paper-only authority.

Chatbot, Workflow, Agent, or Autopilot?

A chatbot responds to a message. A fixed workflow follows a known route. An AI agent adds a goal-directed loop in which a model can choose among explicitly described tools, observe their results, and decide what to do next. In this course, that flexibility is limited to research, interpretation, and routing. Market-data validation, backtest calculations, position sizing, and order constraints remain deterministic Python functions.

An AI trading agent is not an autonomous portfolio manager. It can organise evidence, propose a hypothesis, call approved read-only tools, and explain results. It cannot guarantee a return, invent missing data, loosen a risk limit, or place a live order. Consequential actions remain behind a visible human gate, and the supplied execution code is hard-wired to the paper environment.

Visual framework

Bounded research system: Chat explains; workflow follows fixed rules | Agent chooses among narrow read-only tools | Code calculates and validates deterministically | A person approves any paper order

Unsafe autopilot claim: Model invents or changes trading rules mid-run | One fluent answer is treated as evidence | Keys or live-order endpoints are exposed | No stop condition, audit trail, or owner exists

Seven Building Blocks of a Bounded Trading Agent

A production-minded agent is a system of responsibilities rather than a clever prompt. The goal defines the finish line. Instructions tell the model what evidence it may use and when it must stop. Tools expose narrow capabilities with typed inputs. State preserves verified facts. Guardrails enforce limits outside the model. A human owner resolves ambiguity and authorises the paper-order step.

Weakness in one block propagates. If the market-data tool does not report its feed, the agent cannot judge coverage. If the hypothesis omits a rejection rule, a disappointing result can be rationalised after the fact. If the approval gate is only prose inside the prompt, the model can ignore it. Each block therefore produces observable evidence that another person can inspect.

Visual framework

- Goal — Turn one idea into a traceable research pack, not a buy or sell promise.
- Model — Interprets questions, plans tool use, and drafts explanations; it is not the numeric authority.
- Instructions — Define evidence, output schema, paper-only boundary, stop rules, and escalation.
- Tools — Read market data, validate it, run a backtest, size risk, and prepare or submit a paper ticket.
- State — Versioned JSON, CSV, and logs carry verified facts between labs.
- Guardrails — Allowlisted symbols, bounded dates, quantity caps, paper endpoint, dry run, and approval phrase.
- Human owner — Reviews exceptions, approves the paper ticket, and decides whether research continues.

The Research Agent Loop

The loop begins with a bounded goal such as evaluating a 20/50-day moving-average idea on SPY daily bars. The agent identifies the next missing piece of evidence, calls a tool, reads the returned observation, and checks it against explicit acceptance criteria. Completion is not simply a final paragraph; it is a set of files whose required fields and checks can be verified.

A tool error, empty dataset, conflicting source, failed quality gate, or exceeded iteration limit ends the loop safely. The agent records what happened and asks the named owner to resolve the gap. This stop behaviour is a core capability, not a weakness: continuing with guessed inputs would make every later result look precise while remaining unsupported.

Visual framework

- Receive a bounded research goal
- Plan the next evidence need
- Call one approved tool
- Inspect the observation
- Verify, stop, or continue

A Trading Idea Is Not Yet a Hypothesis

A trading idea is a narrative about why a pattern might exist. A hypothesis turns that narrative into a proposition that historical data can challenge. It names the instrument, bar interval, signal timing, execution assumption, entry, exit, sizing, benchmark, costs, test window, and outcome that would weaken the idea.

The specification must be frozen before the backtest. Otherwise the researcher can change the lookback, date range, asset, or exit rule after seeing results. That is hidden

optimisation. A versioned hypothesis file makes changes visible: version 1.0 is tested as written; a later version requires a stated reason and a fresh out-of-sample check.

Visual framework

Vague idea: Buy when momentum looks strong | Use a good stock and recent data | Exit when conditions change | The strategy should beat the market

Structured hypothesis: SPY daily adjusted bars, fixed date range | 20-day SMA crosses above 50-day SMA after close | Long next session; exit on opposite cross | Compare with buy-and-hold after stated frictions

The HYPER Hypothesis Contract

HYPER is the shared contract used throughout the course. The Hypothesis makes the claim falsifiable. The Yardstick says what will count as useful evidence. Parameters remove ambiguity from the rules. Evidence records sources and data lineage. Risks make failure modes visible before the results create attachment to the idea.

The contract is intentionally machine-readable and human-readable. JSON gives the tools stable fields; the research log explains judgments and sources. The agent may help draft both, but a schema validator checks the structure and a person confirms any field marked OWNER_TO_VERIFY.

Visual framework

- H - Hypothesis — One falsifiable sentence and the market rationale it is intended to test.
- Y - Yardstick — Benchmark, metrics, minimum sample, and rejection criteria defined in advance.
- P - Parameters — Asset, interval, lookbacks, entry, exit, position state, and date range.
- E - Evidence — Source URLs, market-data contract, assumptions, and OWNER_TO_VERIFY items.
- R - Risks — Look-ahead, overfitting, costs, liquidity, regime change, model error, and operational failure.

Research Evidence: Stronger and Weaker Uses

The agent can accelerate discovery and summarisation, but the evidence remains external to the model. Official documentation establishes what an API or simulator actually provides. Dataset metadata shows the feed, time range, and adjustment choice. A second source can reveal a coverage difference or implementation assumption that one provider does not emphasise.

Every source has scope. Alpaca's free IEX feed is useful for initial testing but represents one exchange rather than the full consolidated US market. A daily-bar research exercise

may tolerate that scope if it is recorded. A claim about precise executable prices or liquidity would require more complete quote and venue coverage.

Visual framework

Stronger evidence practice: Exchange, regulator, provider, or official SDK documentation | Direct dataset metadata and timestamped tool output | Independent source used to cross-check a material claim | Citation, access date, scope, and limitation recorded

Weaker evidence practice: Uncited model memory or confident prose | Screenshot with no source, time, or feed | Promotional performance claim treated as proof | Later edits that overwrite the original reasoning

An Agent Output Needs a Contract

Free-form prose hides missing fields. A structured output contract requires the agent to separate evidence from assumptions, identify unknowns, name the next tool, and explain why it has stopped. This makes the output easier to validate and prevents a polished summary from concealing a weak evidence chain.

The course agent writes versioned JSON plus a Markdown log. A schema validator rejects missing keys and wrong types. Deterministic tools consume only the validated fields they need; they do not scrape a narrative response for parameters. This separation reduces ambiguity and makes a later rerun reproducible.

Visual framework

- Claim — What is being proposed, in one falsifiable sentence.
- Known evidence — Source-backed facts and exact tool observations.
- Assumptions — Choices that shape the result but are not measured facts.
- Unknowns — Missing or conflicting items labelled OWNER_TO_VERIFY.
- Next tool — The single approved action needed to reduce uncertainty.
- Stop reason — Why the run cannot safely continue, when applicable.

What a Daily OHLCV Bar Represents

A bar is an aggregation, not the market itself. Provider rules determine which trades are eligible, how sessions are separated, which timezone labels the interval, and whether corporate actions alter historical prices. A strategy that acts after the daily close must not use information that would only have been known during or after the next bar.

Basic consistency checks catch impossible rows: high must be at least the maximum of open and close; low must be at most their minimum; price and volume must be non-negative; timestamps must be ordered and unique. These checks do not prove the data is complete, but failing one is enough to stop the workflow.

Visual framework

- **Timestamp** — The labelled interval and timezone; it must match the intended trading session.
- **Open** — First eligible trade price represented by the bar's aggregation rules.
- **High** — Highest represented trade price; it should not be below open or close.
- **Low** — Lowest represented trade price; it should not be above open or close.
- **Close** — Final represented trade price; adjusted and raw closes answer different questions.
- **Volume** — Aggregated eligible share volume; feed coverage affects the observed value.

Feed Coverage Changes What You Observe

The source name is not a minor metadata field. Alpaca documents that its free IEX feed covers a single exchange, while its SIP feed consolidates activity from US exchanges. Two valid feeds can therefore produce different bars, volumes, and simulated fills. The learner records the feed explicitly and does not compare results as if the inputs were identical.

Fit-for-purpose depends on the question. Daily educational research may begin on the free feed, provided the limitation is disclosed. A high-frequency, liquidity-sensitive, or executable-price claim needs broader and more granular data. The agent can explain the trade-off, but the data owner decides whether coverage is sufficient.

Visual framework

Single-exchange feed: Lower-cost access and useful for initial app testing | Only trades and quotes from the named venue | Volume and intraday prices may differ from the wider market | Scope must be recorded in the data contract

Consolidated feed: Combines eligible activity across US exchanges | Broader price, quote, and volume representation | May require a paid entitlement or provider plan | Still requires timezone, session, and adjustment checks

Time, Session, and Signal Availability

A timestamp is meaningful only with a timezone and session rule. Daily US equity bars are associated with an exchange session, while an API may return UTC timestamps. Converting to local display time without preserving the original zone can shift a bar to the wrong calendar date.

The course strategy computes a crossover from completed daily bars and assumes a next-session execution. Signals are shifted before returns are applied in the backtest. This simple discipline prevents the strategy from receiving the same close that it is supposedly deciding before that close existed.

Visual framework

Known at decision time: Completed bars with unambiguous timezone | Signal computed after the bar closes | Order scheduled for the next eligible session | Calendar and session assumptions recorded

Known only later: Current bar's final close before it occurs | Next session open used to decide today's signal | Revised data silently substituted after the run | Mixed local, exchange, and UTC dates

Corporate Actions and Adjusted Prices

Corporate actions can break a price series if they are ignored. A stock split changes price and shares without creating an equivalent economic loss. Cash dividends contribute to total return. Providers offer adjustment parameters so researchers can choose a consistent series, but the choice must match the strategy and benchmark definition.

Adjusted data is not automatically free from bias. Symbol changes, delistings, and point-in-time membership can still affect a historical universe. This course uses one liquid ETF to keep the beginner exercise bounded, while explicitly teaching why a later multi-asset study needs survivorship-aware constituents and corporate-action records.

Visual framework

- Split — Share count and quoted price change mechanically; an unadjusted series can show a false crash.
- Dividend — Cash distribution affects total return even when the price series alone does not show it.
- Adjustment mode — Raw, split-adjusted, or fully adjusted data must be named in the contract.
- Benchmark consistency — Strategy and benchmark must use comparable return and adjustment assumptions.
- Point-in-time concern — Current symbol mappings and revised histories can leak later knowledge into old dates.
- Verification — Inspect provider metadata and a known event before accepting the series.

The Market-Data Contract

The data contract is the label attached to every dataset. Without it, a CSV of prices cannot be reproduced or judged. The contract records the provider, endpoint, feed, instrument, interval, date range, timezone, session, adjustment mode, transformations, retrieval time, and intended use.

The contract travels with the data into the backtest and audit. If a field changes, the dataset receives a new fingerprint and the old backtest is no longer assumed to apply. This lineage rule is more important than the agent's summary because it connects each result to the actual bytes and assumptions used.

Visual framework

- Provider and endpoint — Who supplied the data and which documented route returned it.
- Feed and entitlement — IEX, SIP, delayed, or another feed plus the account's access scope.
- Instrument — Symbol, asset type, venue context, and currency.
- Time definition — Interval, start, end, timezone, session, and as-of date.
- Transformations — Adjustment, filtering, resampling, missing-value handling, and version.
- Intended use — Educational daily-bar research, not a live executable-price guarantee.

Eight Data-Quality Gates

The validator checks eight categories: required columns and numeric types; ordered unique timestamps; expected date coverage; missing values; OHLC consistency; non-negative volume; freshness relative to the requested end; and a complete provider contract with a file fingerprint. The slide compresses these into five visual stages, while the report records every individual check.

A failed critical gate produces `HOLD_DATA` and prevents the agent from calling the backtest tool. A warning may be acceptable only when its scope is explained and the owner records a decision. For example, a weekend is not a gap in daily equity sessions, while a missing expected trading day needs investigation.

Visual framework

- Schema and types
- Ordered unique timestamps
- Coverage and gaps
- OHLCV consistency
- Freshness and provenance

Worked Example: Freeze the SPY Research Pack

The course scenario tests whether a long-only 20-day versus 50-day simple-moving-average crossover on SPY has useful out-of-sample evidence after stated friction assumptions. The hypothesis does not say the strategy will make money. It says exactly what will be measured and which findings would cause the research owner to hold or revise the idea.

Lab 1 creates the hypothesis and research log. Lab 2 fetches and validates data, then records a SHA-256 fingerprint. The accepted files become immutable inputs to Lab 3. If the hypothesis or data changes, the learner creates a new version instead of overwriting the evidence chain.

Visual framework

- Draft HYPER v1.0
- Human resolves unknowns
- Fetch SPY daily bars
- Run eight quality gates
- Freeze files and fingerprints

Lab 1 — Research an Idea and Freeze a Testable Hypothesis

Learning outcome: LO1 and LO2: operate a bounded tool-using research agent and convert one trading idea into a cited, structured, falsifiable HYPER hypothesis.

Goal: Start the connected SPY research scenario by running an OpenAI Agents SDK research agent with read-only course-source tools and structured output. Separate evidence, assumptions, and unknowns; resolve the required human-owned fields; validate the schema; then freeze version 1.0 with fingerprints before any market data is fetched.

Duration: 50 minutes.

What you'll build

C177-trading-agent-pack/01-hypothesis/ containing hypothesis_spec.json, research_log.md, agent_run_manifest.json, validation_report.json, and fingerprints.json for the frozen SPY 20/50-day moving-average hypothesis (Tools: Python 3.11+, OpenAI Agents SDK, trainer-approved OpenAI API key, Pydantic, scenario brief, approved source notes, text or JSON editor.)

Prerequisites

- Create and activate the repository virtual environment, install labs/resources/requirements.txt, and run labs/resources/preflight.py.
- Store OPENAI_API_KEY only in the local .env file or process environment; never paste it into a prompt, notebook, screenshot, or repository file.
- Read labs/resources/scenario_brief.md and labs/resources/research_sources.md before asking the agent to draft anything.
- Create an empty local folder named C177-trading-agent-pack; do not place real brokerage data or live credentials in it.

Step-by-step

1. Create the Lab 1 output folder and run the preflight check. Continue only when Python and the required packages are ready; an absent OpenAI key must be resolved before the agent run.

```
python labs/resources/workspace.py mkdir C177-trading-agent-pack/01-hypothesis
```

```
python labs/resources/preflight.py --require openai
```

2. Read the scenario without AI. In `research_log.md`, record the fixed course scenario, the human owner, the paper-only boundary, and three questions the evidence must answer.

```
python labs/resources/workspace.py show labs/resources/scenario_brief.md
```

```
python labs/resources/workspace.py copy labs/resources/research_log_starter.md  
C177-trading-agent-pack/01-hypothesis/research_log.md
```

3. Generate a deterministic template first. This creates the expected field structure without calling a model, so you can distinguish schema problems from agent-output problems.

```
python labs/resources/trading_research_agent.py draft --mode template --output  
C177-trading-agent-pack/01-hypothesis/template_hypothesis_spec.json
```

4. Run the bounded research agent. It may call only the supplied read-only source tools and must return the `HypothesisSpec` structured type; it has no market-data or order tool in this lab.

```
python labs/resources/trading_research_agent.py draft --mode agent --output C177-  
trading-agent-pack/01-hypothesis/hypothesis_spec.json --manifest C177-trading-  
agent-pack/01-hypothesis/agent_run_manifest.json
```

5. Compare agent output with the template. Confirm symbol SPY, daily bars, fast window 20, slow window 50, long-only state, next-bar timing, benchmark, fixed date range, adjustment mode, friction, split date, and rejection criteria.

```
python labs/resources/validate_hypothesis.py C177-trading-agent-pack/01-  
hypothesis/hypothesis_spec.json --compare  
C177-trading-agent-pack/01-hypothesis/template_hypothesis_spec.json
```

6. Resolve every `OWNER_TO_VERIFY` item from the scenario and approved sources. Do not silently replace an unknown with a model guess; record the source or human decision in `research_log.md`.

```
python labs/resources/workspace.py locate  
C177-trading-agent-pack/01-hypothesis/hypothesis_spec.json C177-trading-agent-  
pack/01-hypothesis/research_log.md
```

7. Validate the edited hypothesis and the complete research log. The report must show `schema_valid true`, `unresolved_count 0`, `research_log_valid true`, `live_order_requested false`, and status `FROZEN_FOR_DATA` before the next lab.

```
python labs/resources/validate_hypothesis.py C177-trading-agent-pack/01-  
hypothesis/hypothesis_spec.json --research-log C177-trading-agent-pack/01-  
hypothesis/research_log.md --report  
C177-trading-agent-pack/01-hypothesis/validation_report.json
```

8. Audit the evidence table in `research_log.md`. For each material claim, record a URL, access date, exact scope, and limitation; label agent-written prose as a draft rather than a source.

```
python labs/resources/workspace.py unresolved C177-trading-agent-pack/01-hypothesis/research_log.md
```

9. Run the strict frozen-state check, then freeze the Lab 1 checkpoint. Hash the hypothesis, log, manifest, and final validation report; the fingerprint file becomes the integrity reference for Lab 2.

```
python labs/resources/validate_hypothesis.py C177-trading-agent-pack/01-hypothesis/hypothesis_spec.json --research-log C177-trading-agent-pack/01-hypothesis/research_log.md --report C177-trading-agent-pack/01-hypothesis/validation_report.json --require-frozen
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/01-hypothesis --output C177-trading-agent-pack/01-hypothesis/fingerprints.json
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/01-hypothesis --verify C177-trading-agent-pack/01-hypothesis/fingerprints.json
```

10. Review the frozen pack with a partner without modifying it. Point to the exact field that defines signal timing, benchmark, friction, out-of-sample split, rejection criteria, paper-only boundary, and human owner; then verify the unchanged manifest again.

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/01-hypothesis --verify C177-trading-agent-pack/01-hypothesis/fingerprints.json
```

Test it

Run the final validator and fingerprint command. `validation_report.json` must contain `schema_valid=true`, `unresolved_count=0`, `research_log_valid=true`, `research_log_errors=[]`, `live_order_requested=false`, `status=FROZEN_FOR_DATA`, and the expected SPY 20/50 daily rule fields. The log validator must confirm scenario, owner, three evidence questions, five complete source rows, assumptions, resolved unknowns, and the v1.0 UTC record. `fingerprints.json` must list `hypothesis_spec.json`, `research_log.md`, `agent_run_manifest.json`, and `validation_report.json`. A partner must be able to find the signal timing, benchmark, friction, split date, rejection criteria, paper-only boundary, and owner without reading the chat transcript.

Checkpoint for the next lab

Keep the entire 01-hypothesis folder unchanged. Lab 2 verifies its fingerprints, fetches authorised SPY daily bars under the recorded data contract, and writes a separate 02-market-data checkpoint.

Troubleshooting

- The agent run reports a missing or invalid API key: Stop. Confirm `OPENAI_API_KEY` exists in the local process or ignored `.env` file, then rerun preflight. Never print the key or place it in a command argument.

- The agent changes the asset, windows, date range, or paper-only boundary: Discard the draft, reread the scenario brief, and rerun with the supplied instructions. The model may structure the scenario; it may not redesign it.
- Validation finds unresolved fields: Open the exact JSON paths listed in `validation_report.json`, resolve them from an approved source or named human decision, and record that evidence in `research_log.md`.

Challenge

Add a deliberately vague alternative idea to the research log, then write the minimum HYPER fields needed to make it testable without running another backtest.

Reflection

Which field most effectively prevented a fluent trading story from becoming an untestable claim?

Note: Full steps, commands, checkpoints, and troubleshooting are in `labs/lab-01-*.md`. Use only authorised accounts, placeholder or synthetic inputs, and paper-trading credentials. This course does not provide financial advice and includes no live-order path.

Lab 2 — Fetch Market Data and Prove It Is Fit for Purpose

Learning outcome: LO3: connect the workflow to authorised Alpaca market data, record the data contract, run deterministic quality gates, and let the agent interpret only the verified report.

Goal: Verify the frozen hypothesis, connect to Alpaca's historical stock-data client, fetch the declared SPY daily bars, and produce a source contract, CSV, quality report, and fingerprint. Then give the research agent read-only access to the structured quality report so it can recommend `READY_FOR_BACKTEST` or `HOLD_DATA` without changing any numeric observation.

Duration: 60 minutes.

What you'll build

`C177-trading-agent-pack/02-market-data/` containing `spy_daily.csv`, `market_data_contract.json`, `data_quality_report.json`, `data_agent_review.json`, `retrieval_manifest.json`, and `fingerprints.json` (Tools: Alpaca Paper Only account, `alpaca-py` `StockHistoricalDataClient`, `pandas`, deterministic quality validator, OpenAI Agents SDK read-only artifact tool.)

Prerequisites

- Lab 1 is frozen and `C177-trading-agent-pack/01-hypothesis/fingerprints.json` still matches its files.

- ALPACA_API_KEY and ALPACA_SECRET_KEY are stored only in the ignored .env file; ALPACA_PAPER_ONLY remains true.
- The account entitlement and intended feed are understood: the default course request uses the IEX feed and records its single-exchange scope.
- Internet access is available. The synthetic generator is a labelled rejoin fallback and must not be described as real market data.

Step-by-step

1. Create the Lab 2 folder and recheck the Lab 1 fingerprints. Stop if any frozen file changed; restore it or create a documented new version before fetching data.

```
python labs/resources/workspace.py mkdir C177-trading-agent-pack/02-market-data
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/01-hypothesis --verify C177-trading-agent-pack/01-hypothesis/fingerprints.json
```

2. Run preflight for Alpaca credentials. The script checks presence and the paper-only flag without displaying secret values.

```
python labs/resources/preflight.py --require alpaca
```

3. Fetch the exact symbol, dates, interval, adjustment, and feed from the frozen hypothesis. The tool writes raw observations before any agent is asked to interpret them.

```
python labs/resources/fetch_validate_data.py --hypothesis C177-trading-agent-pack/01-hypothesis/hypothesis_spec.json --source alpaca --output-dir C177-trading-agent-pack/02-market-data
```

4. If the provider is unavailable during class, generate the labelled synthetic rejoin checkpoint in a separate folder. Never rename it to hide its source and do not combine it with the Alpaca run.

```
python labs/resources/fetch_validate_data.py --hypothesis C177-trading-agent-pack/01-hypothesis/hypothesis_spec.json --source synthetic --output-dir C177-trading-agent-pack/02-market-data-rejoin
```

5. Inspect market_data_contract.json. Confirm provider, endpoint, feed, entitlement scope, symbol, interval, start, end, timezone, session, adjustment, retrieved_at_utc, intended_use, and CSV fingerprint.

```
python -m json.tool
C177-trading-agent-pack/02-market-data/market_data_contract.json
```

6. Inspect data_quality_report.json. Resolve every critical failure; distinguish an expected market closure from an unexplained long calendar gap, and record any accepted warning with its owner and scope.

```
python -m json.tool
C177-trading-agent-pack/02-market-data/data_quality_report.json
```

7. Spot-check the first three and last three rows. Verify ordered unique timestamps, numeric OHLCV fields, high at least open and close, low at most open and close, and non-negative volume.

```
python labs/resources/inspect_market_data.py C177-trading-agent-pack/02-market-data/spy_daily.csv --head 3 --tail 3
```

8. Run the research agent in read-only data-review mode. It must cite check names and values from the quality report and must not invent a price, fill a gap, or rewrite the status.

```
python labs/resources/trading_research_agent.py review-data --mode agent --pack C177-trading-agent-pack --output C177-trading-agent-pack/02-market-data/data_agent_review.json --manifest C177-trading-agent-pack/02-market-data/agent_review_manifest.json
```

9. Compare the agent review with the deterministic status. If they disagree, the deterministic report controls and the disagreement is recorded as an agent defect.

```
python labs/resources/compare_review.py --report C177-trading-agent-pack/02-market-data/data_quality_report.json --review C177-trading-agent-pack/02-market-data/data_agent_review.json
```

10. Freeze the Lab 2 checkpoint and verify it immediately. Lab 3 must receive the exact CSV whose hash appears in both the contract and quality report.

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/02-market-data --output C177-trading-agent-pack/02-market-data/fingerprints.json
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/02-market-data --verify C177-trading-agent-pack/02-market-data/fingerprints.json
```

Test it

data_quality_report.json must show status READY_FOR_BACKTEST, critical_failures 0, duplicate_timestamps 0, missing_required_values 0, invalid_ohlc_rows 0, negative_volume_rows 0, and the same csv_sha256 recorded in market_data_contract.json. compare_review.py must report REVIEW_ALIGNED. fingerprints.json must verify every Lab 2 artifact. If you used the synthetic rejoin folder, the provider and intended_use fields must visibly identify it as synthetic and the real-data objective remains to be completed.

Checkpoint for the next lab

Keep 02-market-data unchanged. Lab 3 verifies the CSV fingerprint, uses only the accepted dataset, and writes backtest outputs to a new 03-backtest-audit folder.

Troubleshooting

- The API returns an authentication or entitlement error: Confirm you used paper-account API keys and the IEX feed. Do not switch to an undocumented endpoint or

paste keys into the script. Use the labelled synthetic rejoin path while the trainer resolves access.

- The quality report shows HOLD_DATA: Open the critical_checks list, correct the source, date, schema, or corrupted file, and rerun from a clean Lab 2 folder. Never edit prices merely to make the gate green.
- The agent calls the data ready while the report says HOLD_DATA: Keep HOLD_DATA, record an agent-review defect, and rerun the agent after checking the supplied instructions and artifact path.

Challenge

Fetch the same bounded date window from a second authorised feed or provider, record a separate contract, and explain why row counts, volume, or prices can differ without declaring either dataset automatically wrong.

Reflection

Which market-data contract field would be easiest to omit and most damaging to a later interpretation?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-02-*.md. Use only authorised accounts, placeholder or synthetic inputs, and paper-trading credentials. This course does not provide financial advice and includes no live-order path.

Recap — From Trading Idea to Quality Market Data

You can now:

- LO1 and LO2: operate a bounded tool-using research agent and convert one trading idea into a cited, structured, falsifiable HYPER hypothesis
- LO3: connect the workflow to authorised Alpaca market data, record the data contract, run deterministic quality gates, and let the agent interpret only the verified report

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Topic 02 — The Six-Step Trading Workflow and Staying in Control

rule specification | honest backtesting | audit checks | risk sizing | paper orders | verify-then-trust checkpoints

Key concepts

- Rules before results: Entry, exit, timing, sizing, costs, benchmark, and rejection criteria are frozen before the engine runs.
- Deterministic core: Code owns calculations and enforcement; the model interprets, organises evidence, and explains uncertainty.
- Backtest realism: Signals are shifted, data is split, and fees, spread, slippage, fills, and benchmark assumptions are visible.
- Risk budget: Quantity follows a capped account-risk budget and stop distance, never a model's confidence score.
- Paper-only execution: The supplied client uses the paper endpoint, a symbol allowlist, dry run, quantity cap, and explicit approval token.
- Verify then trust: Every stage produces checks, logs, fingerprints, a decision, and a safe rejoin point.

The Six-Step Controlled Trading Workflow

The workflow separates research from execution. First, freeze deterministic rules. Second, acquire data and stop if its contract or quality is inadequate. Third, run the backtest exactly as specified. Fourth, audit leakage, frictions, stability, and the benchmark. Fifth, calculate a position size from a fixed risk budget. Sixth, place a paper-only order after human approval and verify the returned receipt.

The AI agent helps route the work and explain each artifact, but it cannot skip a gate. A HOLD decision ends the current path and records the reason. This state machine prevents a weak backtest from becoming a position-size calculation simply because the conversation continued.

Visual framework

- 1 Define rules
- 2 Acquire and validate data
- 3 Backtest
- 4 Audit
- 5 Size risk
- 6 Paper trade and verify

Rule Specification: Remove Every Hidden Choice

A backtest cannot resolve an ambiguous rule. 'Buy on momentum' leaves the lookback, threshold, timing, order type, and exit undefined. Every unspecified choice becomes an opportunity to tune after seeing the result. The strategy specification removes those hidden degrees of freedom.

For C177, the signal is a 20-day simple moving average crossing the 50-day average on completed adjusted daily closes. The position is either zero or one. The signal is shifted one bar before returns are applied. A buy-and-hold series over the same accepted dates provides the benchmark.

Visual framework

- Universe — One named asset or a point-in-time selection rule.
- Signal — Exact formula, lookback, bar interval, and data field.
- Timing — When the signal becomes known and when the order is assumed to execute.
- Entry and exit — Conditions, position state, and conflict resolution.
- Sizing — Fixed rule independent of model enthusiasm.
- Friction — Fees, spread or slippage, and fill assumptions.
- Benchmark — A comparable alternative such as buy-and-hold.
- Decision — Predeclared evidence for CONTINUE, HOLD, or REVISE.

Look-Ahead Bias: The One-Bar Test

Look-ahead bias occurs when a simulated decision uses information that was not yet available. In vectorised code, a single missing shift can give the strategy today's full return after computing the signal from today's close. The equity curve may look smooth because the backtest is quietly seeing the future.

The course engine makes the timing explicit. It computes the position from completed bars, shifts that position by one row, and only then multiplies it by the next period's return. The learner inspects a small table around one crossover to confirm which timestamp supplies the signal and which timestamp receives the return.

Visual framework

Biased implementation: Today's close creates today's signal | Today's signal receives today's close-to-close return | Final high or low is used before the bar completes | Future revisions influence an earlier decision

Causal implementation: Completed bar creates the signal after close | Signal is shifted before the next return is applied | Execution timing is written in the rule contract | Inputs and code versions are fingerprinted

Backtest Engine: Inputs to Evidence

The deterministic engine loads only a schema-valid strategy specification and a dataset whose fingerprint matches the accepted quality report. It computes moving averages, creates entry and exit transitions, shifts positions, applies friction on turnover, and generates both strategy and benchmark equity curves.

Outputs include metrics JSON, trades CSV, an equity-curve image, and an audit log template. These artifacts are more important than a single headline return. They let a reviewer reproduce the run, inspect individual trades, compare segments, and identify whether a claimed improvement comes from rule logic or a changed assumption.

Visual framework

- Load frozen spec and data
- Create lagged signals
- Apply returns and frictions
- Build trades and equity curves
- Write metrics, plots, and fingerprints

Development Window and Out-of-Sample Window

If the same period is used to invent, tune, and judge a strategy, the result reflects both the market and the researcher's choices. A chronological split gives earlier data to development and later data to out-of-sample review. It does not eliminate overfitting, but it makes the research path more honest.

The learner records the split date before running the engine. Changes inspired by the out-of-sample result create a new hypothesis version and require a new untouched period or a more rigorous walk-forward design. Repeatedly checking the same holdout turns it into another development sample.

Visual framework

Development segment: Used to implement and debug the fixed rule | Can reveal obvious data or code errors | Results may influence design choices | Must not be presented as untouched evidence

Out-of-sample segment: Held back until the specification is frozen | Used once for a cleaner generalisation check | Compared with benchmark and decision criteria | Poor evidence leads to HOLD, not repeated tuning

Read Metrics as a System, Not a Score

No single metric establishes a good strategy. Return without drawdown hides the path. Sharpe ratio compresses assumptions about distribution and volatility. Hit rate can be high while losses are much larger than gains. A small number of trades makes every summary unstable.

The course report presents development, out-of-sample, and full-window metrics beside buy-and-hold. Learners explain what each metric captures, what it omits, and how costs or date choices affect it. The decision is based on a predeclared bundle of evidence rather than whichever number looks most favourable.

Visual framework

- Total return — Growth over the window; it depends strongly on start and end dates.
- Annualised return — Compounded rate normalised for time; unstable with short samples.
- Volatility — Dispersion of returns, not a complete measure of loss risk.
- Sharpe ratio — Excess return per unit of volatility under stated assumptions.
- Maximum drawdown — Largest peak-to-trough decline in the simulated equity curve.

- Turnover — How often exposure changes; it drives friction sensitivity.
- Trade count — A tiny sample cannot support a stable hit-rate story.
- Benchmark gap — Strategy result minus a comparable passive alternative.

Frictionless Results and More Realistic Results

Real orders may fill at prices different from a backtest assumption. QuantConnect describes slippage as the difference between expected and actual fill price and provides models to improve realism. Fees, spread, latency, volume, order type, and market impact also affect the path from signal to fill.

A beginner backtest cannot model every microstructure effect, so the honest choice is to expose a simple friction assumption and test sensitivity. If a small increase in costs erases the result, the evidence is fragile. The report does not hide this behind a single optimised number.

Visual framework

Frictionless simplification: Every order fills immediately at the chosen bar price | No spread, fee, slippage, latency, or market impact | Unlimited liquidity at any quantity | Useful only as an upper-bound diagnostic

Declared reality assumptions: Turnover incurs stated fee and slippage basis points | Execution timing and price field are explicit | Paper limitations and liquidity are disclosed | Sensitivity cases show how conclusions change

Overfitting Red Flags

Overfitting is not only a machine-learning problem. Every choice of asset, date range, indicator, threshold, cost, and exit rule can adapt to noise. Reporting only the best run hides the number of opportunities the researcher had to find it.

The audit records versions, trials, and rejected ideas. It checks nearby parameters for gross instability, but does not use that check to select a new winner during the same run. The correct response to weak out-of-sample evidence is often HOLD and more disciplined research, not another immediate tweak.

Visual framework

- Many trials — Dozens of unreported variants make the best result look rarer than it is.
- Tiny sample — Few trades or one market regime cannot support a stable conclusion.
- Sharp optimum — Only one exact parameter works while nearby values collapse.
- No benchmark — A complex strategy may merely reproduce market exposure.
- Hidden exclusions — Changing assets or dates after inspection creates selection bias.
- Holdout erosion — Repeatedly tuning on the same out-of-sample period destroys its role.

The Backtest Audit Trail

The audit begins by verifying that the strategy, data, and code inputs match the recorded fingerprints. It then inspects rows around a signal change, compares development and out-of-sample segments, checks trade count and drawdown, and reruns declared friction and nearby-parameter cases.

The final decision includes a reason, owner, timestamp, unresolved limitations, and next permissible action. CONTINUE means the research may proceed to a bounded paper-order exercise; it does not mean the strategy is profitable or suitable for live capital. HOLD stops the execution path until the named evidence gap is resolved.

Visual framework

- Recompute fingerprints
- Inspect signal timing
- Compare benchmark and segments
- Stress friction and parameters
- Record CONTINUE, HOLD, or REVISE

Model Work and Deterministic Work

Language models are useful at interpretation and synthesis. They are not reliable enforcement layers and should not be trusted to perform precise financial calculations from prose. The course tools return structured observations; the model explains them without rewriting the numbers.

Controls that protect money, secrets, identity, or auditability live outside the prompt. The paper client ignores any request to use a live endpoint. Quantity is capped by code. The symbol must be allowlisted. Submission remains disabled unless a human supplies the exact approval token after reviewing the generated ticket.

Visual framework

AI model may: Summarise sources and organise a hypothesis draft | Choose the next approved read-only tool | Explain metrics and surface uncertainty | Draft a HOLD or CONTINUE rationale for review

Code or human must: Validate schema, timestamps, prices, and fingerprints | Compute signals, returns, costs, and position size | Enforce paper endpoint, allowlist, caps, and dry run | Approve the ticket and own the decision

Position Size Comes from Risk Budget and Stop Distance

CME's educational guidance links position size to two inputs: where the stop is placed and how much of the account the trader is willing to risk. The course formula is $\text{quantity} = \text{floor}(\text{risk budget} / \text{per-share risk})$, then the result is capped by maximum position value and a hard course quantity limit.

A stop order cannot guarantee the planned loss because markets can gap and fills can slip. The calculation is therefore a planning bound, not a promise. The paper ticket records entry reference, stop reference, timestamp, risk fraction, exposure cap, final quantity, and every cap that changed the raw result.

Visual framework

- Account equity — The paper account value used only for the exercise.
- Risk fraction — A small predeclared fraction chosen by the human owner, not the agent.
- Risk budget — Account equity multiplied by risk fraction.
- Per-share risk — Absolute difference between planned entry and protective stop.
- Risk quantity — Floor of risk budget divided by per-share risk.
- Exposure cap — A second ceiling based on maximum position value and available buying power.

Worked Position-Size Example

With paper equity of \$100,000 and a 0.50% risk fraction, the risk budget is \$500. If the planned entry reference is \$500 and the stop reference is \$490, per-share risk is \$10 and the raw risk quantity is 50 shares. A 10% exposure cap also permits at most 20 shares at a \$500 reference price, so the final quantity becomes 20.

The example shows why risk and exposure are different. The stop distance controls the planned loss per share, while the exposure cap limits how much account value is concentrated in one symbol. The smallest applicable bound wins, and the ticket records which bound was active.

Visual framework

- Paper equity = \$100,000
- Risk fraction = 0.50%
- Entry \$500; stop \$490
- Raw quantity = $\text{floor}(\$500 / \$10) = 50$
- Apply exposure and course caps

The Pre-Trade Paper Ticket

The ticket is a proposal, not an order. It consolidates the exact symbol, side, quantity, order type, time in force, reference price, environment, evidence fingerprints, and active caps. A person can review one artifact instead of reconstructing the decision from a chat transcript.

The supplied tool generates the ticket in dry-run mode by default. It refuses live mode, non-allowlisted symbols, non-positive or excessive quantities, stale tickets, missing audit decisions, and missing approval. The agent may explain a refusal but cannot override it.

Visual framework

- Instrument — Allowlisted symbol, asset type, side, and paper environment.
- Evidence — Accepted hypothesis, data, backtest, audit decision, and fingerprints.
- Quantity — Risk calculation, exposure cap, buying-power check, and hard maximum.
- Order — Type, time in force, reference price, and paper endpoint.
- Limits — No live route, no secret in logs, no unsupported symbol, no blank approval.
- Owner — Reviewer, approval time, approval token, and reason for the exercise.

Paper Trading Is Useful and Incomplete

Alpaca describes paper trading as a real-time simulation and explicitly lists differences from live trading, including market impact, information leakage, latency slippage, queue position, price improvement, regulatory fees, and dividends. A paper fill is therefore evidence that the software path worked under simulator assumptions, not that a live order would receive the same result.

The course stays paper-only. Learners verify that the returned account and order belong to the paper environment, save the order identifier and status, and then cancel an open exercise order when appropriate. No live credentials, endpoints, or live-order instructions are included.

Visual framework

What paper trading exercises: Authentication, request schema, account checks, and order lifecycle | Real-time data handling and software error paths | Logging, approval, idempotency, and receipt verification | Operational rehearsal without risking capital

What it may not reproduce: Market impact, information leakage, or latency slippage | Queue position and realistic available liquidity | All fees, dividends, price improvement, or live behaviour | Future profitability or suitability for real money

Human Gate and Safe Submission

The human gate occurs after the deterministic ticket exists. The reviewer checks the paper environment, symbol, quantity, side, order type, active risk cap, accepted audit decision, and absence of secrets. Only then is the exact approval token added to a local command that is not stored in the repository.

Submission uses a unique client order ID so a retry can be reconciled rather than blindly duplicated. If the network response is uncertain, the tool queries by client order ID before attempting anything else. The safe response to ambiguity is to stop and inspect the paper dashboard.

Visual framework

- Generate dry-run ticket
- Review evidence and limits
- Human enters exact approval token
- Submit to paper endpoint

- Verify receipt or stop

Verify-Then-Trust Checkpoints

Verify-then-trust is a habit of requiring observable evidence before the next capability is enabled. It does not mean verifying once and trusting forever. Model, prompt, code, data, account, or provider changes can invalidate prior evidence and require a rerun.

NIST's AI risk guidance emphasises testing, evaluation, verification, validation, documentation, and clear human-AI roles. The course turns those ideas into six concrete gates. Each gate has a file, a check, an owner, a decision, and a rejoin checkpoint so a learner can recover without inventing missing state.

Visual framework

- Hypothesis gate — Schema valid, assumptions separated, unknowns owned, version frozen.
- Data gate — Contract complete, quality checks recorded, fingerprint accepted.
- Backtest gate — Timing causal, benchmark present, segments and frictions reported.
- Audit gate — Leakage, trials, stability, limitations, and decision documented.
- Risk gate — Quantity recomputed from approved inputs and bounded by caps.
- Execution gate — Paper endpoint, human approval, unique ID, receipt, and reconciliation.

Failure, Pause, Reconcile, and Roll Back

A reliable workflow is designed around failure paths. APIs time out, providers revise data, credentials expire, models change, and simulated orders can behave unexpectedly. The agent must expose the uncertainty and stop instead of smoothing it into a confident narrative.

Rollback is practical because each lab produces versioned files. The operator can disable submission, cancel an open paper order, restore the last verified specification and code, and rerun from a known checkpoint. Logs and fingerprints show exactly which state was active when the issue occurred.

Visual framework

- Tool failure — Record the error; do not fabricate a result or continue from stale state.
- Data drift — Freeze the new dataset separately and rerun quality and backtest evidence.
- Duplicate risk — Query the client order ID before retrying a timed-out request.
- Unexpected fill — Stop the workflow, inspect the paper account, and preserve the receipt.
- Model change — Rerun stable prompts and tool-choice checks before accepting explanations.

- Rollback — Cancel open paper orders, disable submission, and restore the last verified version.

Lab 3 — Backtest the Rules and Audit the Evidence

Learning outcome: LO4: run a causal deterministic backtest, compare a benchmark and holdout, stress stated assumptions, and record an auditable research decision.

Goal: Verify the accepted hypothesis and market-data fingerprints, run the fixed 20/50-day crossover engine with a one-bar signal shift and declared frictions, then inspect trades, development and out-of-sample metrics, benchmark results, drawdown, and sensitivity checks. The agent receives only read-only report tools and drafts a review that the human auditor must reconcile with deterministic checks.

Duration: 65 minutes.

What you'll build

C177-trading-agent-pack/03-backtest-audit/ containing strategy_spec.json, signal_sample.csv, trades.csv, equity_curve.csv, equity_curve.png, backtest_metrics.json, sensitivity_report.json, audit_log.md, backtest_agent_review.json, run_manifest.json, and fingerprints.json (Tools: pandas, NumPy, Matplotlib, deterministic C177 backtest engine, OpenAI Agents SDK read-only artifact review, Markdown editor.)

Prerequisites

- Lab 1 and Lab 2 fingerprints verify and the deterministic market-data status is READY_FOR_BACKTEST.
- The CSV hash matches both market_data_contract.json and data_quality_report.json.
- The hypothesis remains version 1.0; parameter or date changes require a separately documented version rather than an overwrite.
- The learner understands that historical and out-of-sample results do not guarantee future behaviour.

Step-by-step

1. Create the Lab 3 folder and verify both upstream checkpoints. Stop on any mismatch instead of accepting a stale or edited input.

```
python labs/resources/workspace.py mkdir C177-trading-agent-pack/03-backtest-audit
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/01-hypothesis --verify C177-trading-agent-pack/01-hypothesis/fingerprints.json
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/02-market-data --verify C177-trading-agent-pack/02-market-data/fingerprints.json
```

- 2.** Run the deterministic engine from the frozen specification and accepted CSV. Do not ask the model to calculate returns, drawdown, Sharpe ratio, trades, or quantity.

```
python labs/resources/backtest_and_audit.py --hypothesis
C177-trading-agent-pack/01-hypothesis/hypothesis_spec.json --data C177-trading-
agent-pack/02-market-data/spy_daily.csv --quality-report
C177-trading-agent-pack/02-market-data/data_quality_report.json --output-dir C177-
trading-agent-pack/03-backtest-audit
```

- 3.** Inspect `signal_sample.csv` around at least one crossover. Confirm the raw signal is computed from completed bars, `position_used` is shifted by one row, and the same row's return never receives an unshifted signal.

```
python labs/resources/inspect_signal_timing.py C177-trading-agent-pack/03-
backtest-audit/signal_sample.csv
```

- 4.** Read `backtest_metrics.json` as a system. Compare full, development, and out-of-sample strategy metrics with buy-and-hold; record trade count, total return, annualised return, volatility, Sharpe ratio, maximum drawdown, turnover, and benchmark gap.

```
python -m json.tool
C177-trading-agent-pack/03-backtest-audit/backtest_metrics.json
```

- 5.** Open `equity_curve.png` and `trades.csv`. Find the largest drawdown period and inspect at least two entry-exit sequences; confirm the trade rows reconcile with changes in the position column.

```
python labs/resources/workspace.py locate C177-trading-agent-pack/03-backtest-
audit/equity_curve.png C177-trading-agent-pack/03-backtest-audit/trades.csv
```

- 6.** Inspect `sensitivity_report.json`. Compare the declared friction case with zero and higher friction, then compare nearby 15/50 and 25/50 windows without selecting a new winner during this run.

```
python -m json.tool
C177-trading-agent-pack/03-backtest-audit/sensitivity_report.json
```

- 7.** Complete the human audit log. Record leakage check, data lineage, trial count, benchmark, split date, cost assumptions, parameter stability, limitations, and a `CONTINUE_RESEARCH`, `HOLD`, or `REVISE` decision with reasons.

```
python labs/resources/workspace.py copy labs/resources/backtest_audit_starter.md
C177-trading-agent-pack/03-backtest-audit/audit_log.md
```

```
python labs/resources/workspace.py locate C177-trading-agent-pack/03-backtest-
audit/audit_log.md
```

- 8.** Run the agent in read-only backtest-review mode. It must cite report paths and values, preserve the deterministic status, and name limitations instead of recommending a trade.

```
python labs/resources/trading_research_agent.py review-backtest --mode agent --
pack C177-trading-agent-pack --output
```

```
C177-trading-agent-pack/03-backtest-audit/backtest_agent_review.json --manifest  
C177-trading-agent-pack/03-backtest-audit/agent_review_manifest.json
```

9. Reconcile human, agent, and deterministic decisions. A disagreement is documented; it is never resolved by rewriting metrics. Lab 4 is enabled only when the deterministic checks allow `CONTINUE_RESEARCH` and the human audit agrees.

```
python labs/resources/compare_review.py --report C177-trading-agent-pack/03-  
backtest-audit/backtest_metrics.json --review C177-trading-agent-pack/03-backtest-  
audit/backtest_agent_review.json --human C177-trading-agent-pack/03-backtest-  
audit/audit_log.md
```

10. Freeze and verify the Lab 3 checkpoint. The audit decision permits only a bounded paper exercise; it is not a statement of expected return or suitability for live capital.

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/03-backtest-  
audit --output C177-trading-agent-pack/03-backtest-audit/fingerprints.json
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/03-backtest-  
audit --verify C177-trading-agent-pack/03-backtest-audit/fingerprints.json
```

Test it

The run manifest must show matching hypothesis and data fingerprints. `signal_sample.csv` must demonstrate a one-row position shift. `backtest_metrics.json` must contain full, development, out_of_sample, and benchmark sections plus `deterministic_status`. `sensitivity_report.json` must include at least three friction cases and the 15/50, 20/50, and 25/50 parameter cases. `compare_review.py` must report the deterministic, agent, and human decisions explicitly. All Lab 3 fingerprints must verify.

Checkpoint for the next lab

Keep the verified 03-backtest-audit folder. Lab 4 reads the accepted audit decision and latest data reference, calculates a bounded quantity, generates a dry-run ticket, and submits only after the explicit human paper-approval gate.

Troubleshooting

- The engine reports an input fingerprint mismatch: Restore the exact frozen file or create a new version and rerun its upstream gate. Do not edit the manifest to match a changed file.
- The signal-timing inspection shows the position was not shifted: Stop the run, inspect the `position_used` construction, and rerun from a clean output folder. Any metrics from the biased run are invalid.
- Out-of-sample evidence or friction sensitivity is weak: Record `HOLD` or `REVISE`. Preserve the result; do not tune on the same holdout and then relabel it as untouched evidence.

Challenge

Add a walk-forward report with at least three chronological windows while preserving the original v1.0 run and trial log; explain what the extra windows reveal and what they still cannot prove.

Reflection

Which audit check changed your interpretation of the headline return most, and why?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-03-*.md. Use only authorised accounts, placeholder or synthetic inputs, and paper-trading credentials. This course does not provide financial advice and includes no live-order path.

Lab 4 — Size Risk, Approve, and Verify a Paper Trade

Learning outcome: LO5 and LO6: calculate a bounded quantity, generate and review a paper-order ticket, apply the human gate, submit only to simulation, and verify or roll back the result.

Goal: Use the accepted research evidence to calculate quantity from a human-set paper risk budget, stop distance, exposure ceiling, buying power, and hard course cap. Generate a dry-run ticket, let the agent perform a read-only evidence review, obtain explicit human approval, submit through an Alpaca TradingClient constructed with paper=True, reconcile the receipt, and close with a secret-free final manifest.

Duration: 70 minutes.

What you'll build

C177-trading-agent-pack/04-paper-trade/ containing risk_calculation.json, paper_order_ticket.json, ticket_agent_review.json, human_approval.md, submission_attempt.json, a paper receipt or structured error, reconciliation_log.md, rollback_record.md, final_manifest.json, and fingerprints.json (Tools: Alpaca TradingClient in paper mode, deterministic C177 risk and order guardrails, OpenAI Agents SDK read-only ticket review, paper dashboard, text editor.)

Prerequisites

- Lab 3 fingerprints verify, deterministic_status permits CONTINUE_RESEARCH, and the human audit explicitly permits only a bounded paper exercise.
- ALPACA_PAPER_ONLY=true and paper-account keys are present locally; no live credentials are available to the course script.
- The reviewer understands that a stop reference and paper fill do not guarantee a live loss bound or future performance.
- The paper account is suitable for the exercise and the course allowlist contains only SPY.

Step-by-step

1. Create the Lab 4 folder and verify the Lab 3 checkpoint. Read the audit decision and stop immediately unless both the deterministic and human records permit a paper-only exercise.

```
python labs/resources/workspace.py mkdir C177-trading-agent-pack/04-paper-trade
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/03-backtest-audit --verify C177-trading-agent-pack/03-backtest-audit/fingerprints.json
```

```
python labs/resources/paper_trade.py check-gate --pack C177-trading-agent-pack
```

2. Generate the deterministic risk calculation and dry-run ticket. Use 0.50% paper-account risk, a 2% stop-distance exercise, a 10% exposure cap, and the course hard cap of 20 shares unless the trainer supplies smaller limits.

```
python labs/resources/paper_trade.py prepare --pack C177-trading-agent-pack --risk-fraction 0.005 --stop-percent 0.02 --max-exposure-fraction 0.10 --hard-max-qty 20
```

3. Recompute the quantity manually from the displayed paper equity, entry reference, stop reference, risk budget, per-share risk, exposure cap, buying-power cap, and hard cap. Record the smallest active bound in human_approval.md.

```
python labs/resources/workspace.py copy labs/resources/human_approval_starter.md C177-trading-agent-pack/04-paper-trade/human_approval.md
```

```
python -m json.tool C177-trading-agent-pack/04-paper-trade/risk_calculation.json
```

4. Inspect paper_order_ticket.json. Confirm environment PAPER, symbol SPY, side BUY, positive quantity no greater than 20, order type MARKET, time in force DAY, unique client_order_id, evidence fingerprints, and submission_enabled false.

```
python -m json.tool C177-trading-agent-pack/04-paper-trade/paper_order_ticket.json
```

5. Run the agent in read-only ticket-review mode. It may cite evidence and missing conditions; it cannot change quantity, enable submission, enter approval, call Alpaca, or recommend live use.

```
python labs/resources/trading_research_agent.py review-ticket --mode agent --pack C177-trading-agent-pack --output C177-trading-agent-pack/04-paper-trade/ticket_agent_review.json --manifest C177-trading-agent-pack/04-paper-trade/agent_review_manifest.json
```

6. Complete the human review. Enter reviewer initials, UTC time, paper environment, exact quantity, active cap, evidence decision, and APPROVE_PAPER_EXERCISE or HOLD. Do not store an API key or the command approval token in this file.

```
python labs/resources/workspace.py locate C177-trading-agent-pack/04-paper-trade/human_approval.md
```

7. Preview the exact request without submitting. The output must say DRY_RUN, show the paper endpoint, and refuse if any gate is missing.

```
python labs/resources/paper_trade.py submit --pack C177-trading-agent-pack --dry-run
```

8. After the reviewer authorises the exercise, submit once with the exact ephemeral approval token. The script constructs `TradingClient(..., paper=True)`, verifies the paper account, uses the saved `client_order_id`, and writes the receipt without secrets.

```
python labs/resources/paper_trade.py submit --pack C177-trading-agent-pack --approve C177-PAPER-APPROVED
```

9. Verify the returned order through the API and paper dashboard. Record order ID, client order ID, symbol, side, quantity, status, submitted time, and any fill information in the receipt and reconciliation log.

```
python labs/resources/paper_trade.py verify --pack C177-trading-agent-pack
```

10. Test idempotency by rerunning the dry run. It must find the existing `client_order_id` or receipt and refuse to create a duplicate submission.

```
python labs/resources/paper_trade.py submit --pack C177-trading-agent-pack --dry-run
```

11. If the exercise order remains open, request cancellation and let the script poll for a bounded time until the API confirms a terminal state. A still-open status remains on HOLD. If it filled, inspect or close the exercise position under trainer direction, then rerun rollback with `--confirm-position-reviewed` so the resolved state is explicit.

```
python labs/resources/paper_trade.py rollback --pack C177-trading-agent-pack
```

12. Build the final manifest, scan for secret-shaped strings, freeze the folder, and verify every artifact. Any secret hit must be removed and the affected paper key rotated before sharing.

```
python labs/resources/finalise_pack.py C177-trading-agent-pack
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/04-paper-trade --output C177-trading-agent-pack/04-paper-trade/fingerprints.json
```

```
python labs/resources/fingerprint_pack.py C177-trading-agent-pack/04-paper-trade --verify C177-trading-agent-pack/04-paper-trade/fingerprints.json
```

Test it

`risk_calculation.json` must show every formula input and the final quantity as the minimum applicable cap. `paper_order_ticket.json` must show environment PAPER, symbol SPY, quantity at most 20, `submission_enabled` false before approval, and a unique `client_order_id`. `submission_attempt.json` must exist before a real API call. `paper_order_receipt.json` must reconcile the same client order ID, or `paper_submission_error.json` plus `reconciliation_log.md` must confirm that no order exists without fabricating success. `rollback_record.md` must show Resolved YES.

final_manifest.json must report secret_hits 0, no release defects, and verified checkpoints from all four labs.

Checkpoint for the next lab

Retain the complete C177-trading-agent-pack as a paper-only research record. Rerun all six gates whenever the hypothesis, prompt, model, source, feed, data, code, risk limit, account, or provider changes.

Troubleshooting

- The gate checker refuses to prepare a ticket: Read the exact missing decision or fingerprint path. Restore and verify the upstream evidence; never edit the checker or ticket to bypass the gate.
- The submit command reports an uncertain timeout: Do not submit again. The saved submission_attempt.json contains the client_order_id. Run verify; it queries that exact ID and records a receipt, CONFIRMED_NO_ORDER result, or unresolved reconciliation before any further action.
- A secret scan reports a possible key: Stop sharing, remove the value from every artifact and git history if needed, rotate the affected paper key, then rerun the scan and fingerprints.

Challenge

Add a trainer-approved limit-order dry-run path with a maximum ticket age, price-deviation check, and cancellation timeout while preserving the paper-only endpoint and human gate.

Reflection

Which control would still protect the paper account if the model produced a confident but incorrect recommendation?

Note: Full steps, commands, checkpoints, and troubleshooting are in labs/lab-04-*.md. Use only authorised accounts, placeholder or synthetic inputs, and paper-trading credentials. This course does not provide financial advice and includes no live-order path.

Recap — The Six-Step Trading Workflow and Staying in Control

You can now:

- LO4: run a causal deterministic backtest, compare a benchmark and holdout, stress stated assumptions, and record an auditable research decision
- LO5 and LO6: calculate a bounded quantity, generate and review a paper-order ticket, apply the human gate, submit only to simulation, and verify or roll back the result

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Wrap-Up - Your Controlled Trading Research Pack

You have moved one deliberately simple trading idea through an auditable chain from hypothesis to market data, backtest, risk calculation, and paper-only order evidence.

What the agent contributed

- Converted a broad idea into a structured draft and exposed unknowns.
- Selected approved tools and explained their structured observations.
- Maintained a clear separation between claims, evidence, assumptions, and stop reasons.

What deterministic controls contributed

- Validated schemas, data quality, fingerprints, signal timing, metrics, and quantity.
- Enforced symbol, risk, exposure, quantity, paper-environment, approval, and idempotency limits.
- Produced reproducible files that can be reviewed independently of the chat session.

What remains uncertain

- Historical and out-of-sample results do not guarantee future performance.
- Simple friction assumptions and a paper simulator do not reproduce the live market.
- One asset, one rule, and one period are a learning exercise, not an investment conclusion.

Next Steps

- Re-run the full pack from an empty output folder and compare the new fingerprints and logs.
- Replace only one declared parameter, create hypothesis version 1.1, and preserve version 1.0 for comparison.
- Add walk-forward windows and broader point-in-time data before considering a multi-asset research claim.
- Review OpenAI Agents SDK, Alpaca, NIST, CME, and backtest-engine documentation for changes before each delivery.
- Keep every experiment paper-only until an appropriately qualified owner establishes a separate governance, compliance, risk, and operational process.

Glossary

- **AI agent** — A goal-directed system in which a model can choose among approved tools, observe results, and continue or stop within defined limits.
- **Backtest** — A simulation that applies fixed rules to historical data under stated timing, cost, and fill assumptions.
- **Benchmark** — A comparable reference, such as buy-and-hold, used to interpret whether complexity added evidence.
- **Corporate-action adjustment** — A transformation that accounts for events such as splits or dividends in a historical price series.
- **Data contract** — Metadata that records a dataset's provider, feed, instrument, interval, dates, timezone, adjustments, transformations, and intended use.
- **Drawdown** — A peak-to-trough decline in an equity curve before a new peak is reached.
- **Fingerprint** — A cryptographic hash used to identify the exact bytes of an input or output file.
- **HYPER** — The course hypothesis contract: Hypothesis, Yardstick, Parameters, Evidence, and Risks.
- **Look-ahead bias** — Use of information in a simulated decision before that information would have been available.
- **Market-data feed** — A defined source and coverage of trades, quotes, or bars, such as a single venue or consolidated market feed.
- **Out-of-sample** — A later period held apart from rule development and used for a cleaner review of generalisation.
- **Paper trading** — A simulated order environment that exercises software and operations without routing an order to a live exchange.
- **Per-share risk** — The absolute difference between planned entry and stop reference used in the course sizing formula.
- **Position sizing** — Calculation of quantity from a human-set risk budget, stop distance, exposure cap, and other limits.
- **Slippage** — The difference between the expected order price and the actual fill price.
- **Verify-then-trust** — A recurring practice of requiring observable evidence at each boundary before enabling the next capability.